

Tutoría Fundamentos de Informática

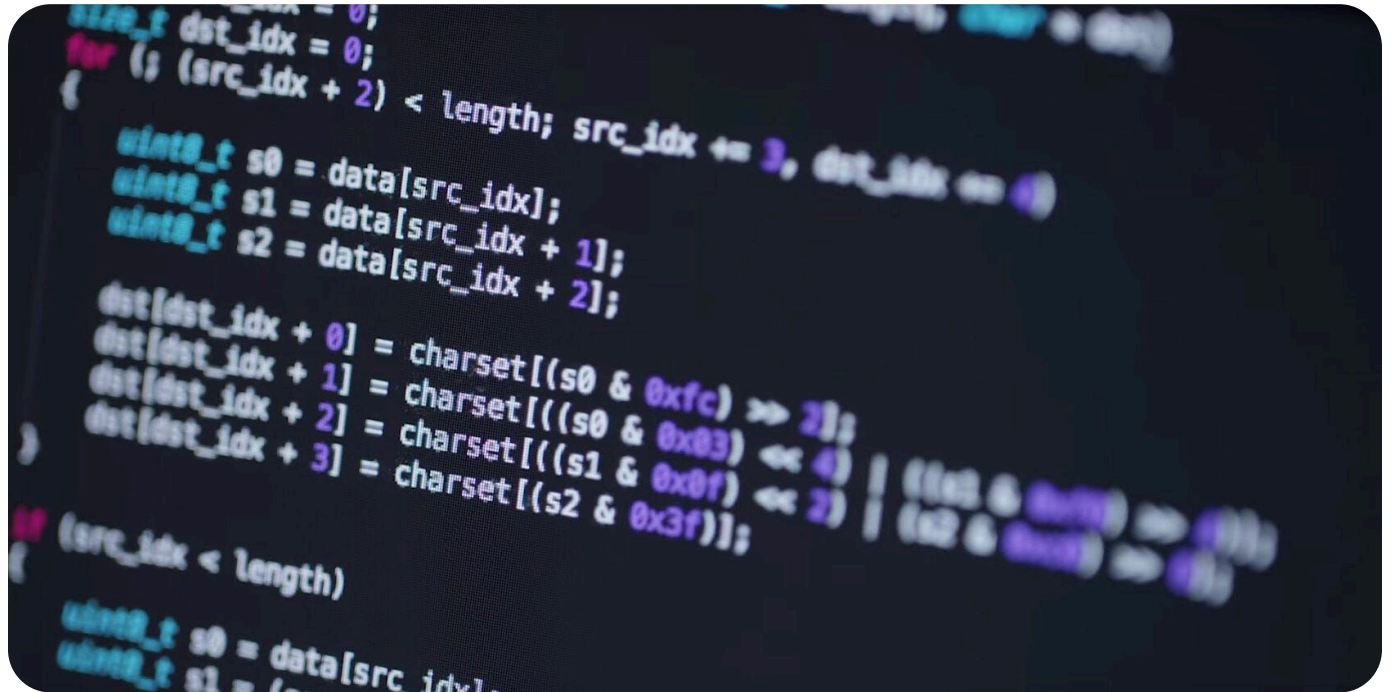
Estructuras iterativas - ciclos



¿Qué son los ciclos?

Estructuras de control que repiten un bloque de código N cantidad de veces

También conocidos como **bucles** o **estructuras condicionales**



```
size_t dst_idx = 0;
for (; (src_idx + 2) < length; src_idx += 3, dst_idx += 4)
{
    uint8_t s0 = data[src_idx];
    uint8_t s1 = data[src_idx + 1];
    uint8_t s2 = data[src_idx + 2];

    dst[dst_idx + 0] = charset[(s0 & 0xfc) >> 2];
    dst[dst_idx + 1] = charset[((s0 & 0x03) << 4) | ((s1 & 0xf0) >> 4)];
    dst[dst_idx + 2] = charset[((s1 & 0x0f) << 2) | ((s2 & 0xc0) >> 2)];
    dst[dst_idx + 3] = charset[(s2 & 0x3f)];
}

if (src_idx < length)
{
    uint8_t s0 = data[src_idx];
    uint8_t s1 = data[src_idx + 1];
```

¿Para qué sirven?



Repetir tareas automáticamente



Recorrer listas y arreglos



Procesar datos



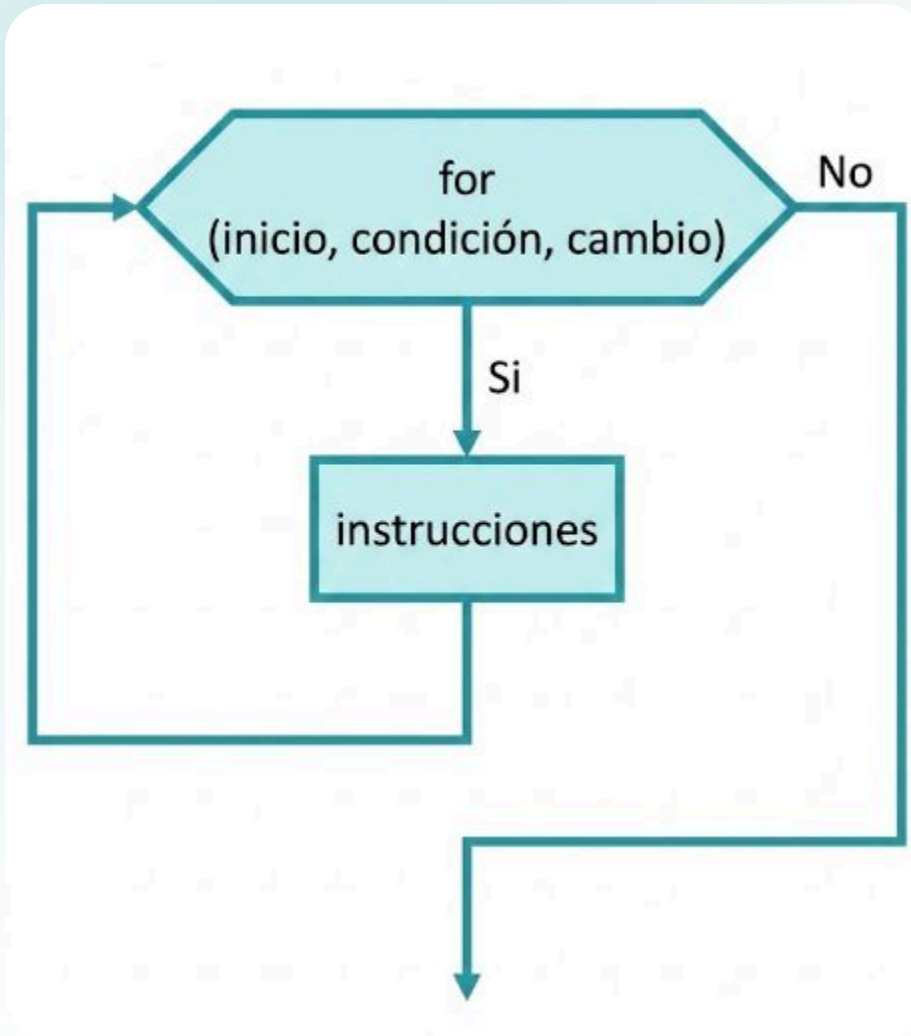
Ejecutar hasta cumplir condición

Partes de un ciclo

- **Inicialización:** se ejecuta una sola vez al inicio del ciclo, en este paso se asignan los valores iniciales a las variables que forman parte del ciclo
- **Condición:** sirve para evaluar la continuación o finalización de las iteraciones
- **Instrucciones:** es el conjunto de acciones/instrucciones que vamos a ejecutar en cada iteración
- **Actualización:** modificación de las variables que participan en el bloque de instrucciones o de la condición del ciclo

Ciclo **for**

- Lo utilizamos cuando **queremos repetir instrucciones** debemos repetir una acción en nuestro código
- Usamos una variable de control/contador que lleva el seguimiento de las iteraciones
- Las repeticiones continúan mientras **la condición del ciclo sea** verdadera



Ejemplo ciclo **for**:

Sintaxis

```
for(inicialización; condición; actualización) {  
    // instrucciones a repetir  
}
```

Ejemplo en C++

- Imprimir números del 1 al 100

```
for(int i = 1; i <= 100; i++) {  
    cout << i << endl;  
}
```



Ciclo **while**

- Cuando dependemos de una condición para **repetir** una acción
- Primero verifica la condición y solo ejecuta las instrucciones si es verdadera
- Las repeticiones continúan hasta que **la condición deje de cumplirse**

Ejemplo: Ciclo **while**

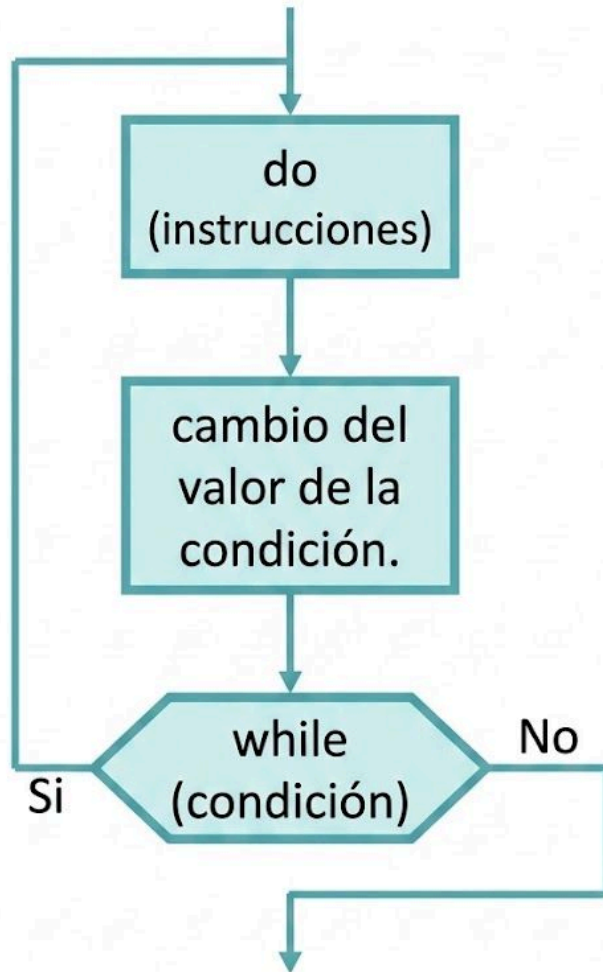
Sintaxis

```
while(condición) {  
    // instrucciones a repetir  
    // actualizacion de la variable  
}
```

Ejemplo en C++

- Imprimir números desde n hasta 1

```
int n;  
cin >> n;  
while(n > 0) {  
    cout << n << endl;  
    n--;  
}
```

Ciclo do-while

- El bloque de instrucciones **se ejecuta al menos una vez**, ya que la condición se verifica después de ejecutarlo
- Si la condición se cumple, el bloque **vuelve a ejecutarse**
- La repetición termina cuando la condición deja de cumplirse

Ejemplo: Ciclo do-while

Sintaxis

```
do {  
    // instrucciones a repetir  
} while(condición);
```

Ejemplo en C++

```
do {  
    cout << "Ingrese un numero positivo: ";  
    cin >> numero;  
} while(numero <= 0);
```

Palabras clave en ciclos



break

- Esta se utiliza para interrumpir el ciclo y continuar con la siguiente instrucción fuera de este
- En otras palabras: "Ya no necesito seguir repitiendo"



Ejercicio #2. Parcial I - I Ciclo 2026

Objetivo:

Implemente la función **verificaDigitosIguales()**, la cual reciba como parámetro un número entero positivo y determine si todos sus dígitos son iguales.

- Ejemplo:

1, 222, 9999

La función debe retornar true o false según corresponda



Ejercicio #1. Parcial I - I Ciclo 2025

Objetivo:

Escriba un programa que solicite al usuario un número entero positivo **n** y muestre la suma de los primeros **n** términos de la sucesión de Fibonacci que sean números primos.

La sucesión de Fibonacci es una secuencia de números en la que cada término es la suma de los dos términos anteriores. Los dos primeros términos de la sucesión siempre son 0 y 1.

Ejemplo: Los primeros 12 términos de la sucesión de Fibonacci son:

0, 1, **2, 3, 5, 8, 13**, 21, 34, 55, **89**, 144

De la secuencia anterior, solo los términos subrayados y en negrita son los números primos que se deben sumar. Para este ejemplo, se debe mostrar en pantalla **112**, pues $2+3+5+13+89 = 112$.

Indicaciones:

- Escriba una función **esPrimo(n)** para verificar si un número es primo o no. Esta función debe retornar verdadero si el número **n** recibido es primo, o false, si no. (15 pts.)
- Escriba una función **fibonacci(n)** que genere los primeros **n** términos de la sucesión de Fibonacci, sume los términos que sean números primos y devuelva el resultado de la suma. (15 pts.)
- En el main() lea del usuario el valor **n** y muestre el resultado de la suma descrita. (5 pts.)